Министерство Образования, Культуры и Исследований Республики Молдова

Технический Университет Молдовы

Факультет Вычислительной Техники и Микроэлекроники

Департамент Программной Инженерии

Отчет

по лабораторной работе № 1

по дисциплине

Программирование мобильных приложений

Тема: «**Среда разработки, язык Dart и выбор темы**»

Выполнил: ст. гр. TI-235

Davîdov Iaroslav

Проверил: Leah Arcadie

Кишинев 2026

Цель работы

Настроить среду разработки Flutter для Android, проверить запуск демонстрационного приложения, освоить основные средства языка Dart и определить тему сквозного проекта курса.

Часть А Установка и проверка среды

Использована macOS на архитектуре arm64. Установлены Flutter 3.47.5, Android SDK 36.0.0 и Dart 3.13.4 из Homebrew. Для запуска выбран Android-эмулятор с API 35. Указанные версии соответствуют минимальным требованиям задания Flutter 3.35+ и Dart 3.9+.

Результат команды flutter doctor

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.5, on macOS 26.6.2 25G83 darwin-arm64, locale
ru-MD)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[!] Xcode - develop for iOS and macOS
    ✗ Xcode installation is incomplete; a full installation is necessary for iOS
and macOS development.
      Download at: https://developer.apple.com/xcode/
      Or install Xcode via the App Store.
      Once installed, run:
        sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
        sudo xcodebuild -runFirstLaunch
    ! CocoaPods not installed.
        CocoaPods is a package manager for iOS or macOS platform code.
        Without CocoaPods, plugins will not work on iOS or macOS.
        For more info, see https://flutter.dev/to/platform-plugins
      For installation instructions, see
https://guides.cocoapods.org/using/getting-started.html#installation
[✗] Chrome - develop for the web (Cannot find Chrome executable at
/Applications/Google Chrome.app/Contents/MacOS/Google Chrome)
    ! Cannot find Chrome. Try setting CHROME_EXECUTABLE to a Chrome executable.
[✓] Connected device (2 available)
[✓] Network resources

! Doctor found issues in 2 categories.
```

Android toolchain и подключённое Android-устройство отмечены как готовые. Предупреждения по Xcode/CocoaPods и
Chrome сохранены в полном выводе: iOS/macOS и веб в данном Android-сценарии не проверялись. Лицензии Android SDK
приняты пользователем; соответствующее предупреждение устранено.

Результат команды flutter devices

```
Found 2 connected devices:
  sdk gphone64 arm64 (mobile) • emulator-5554 • android-arm64 • Android 15 (API
35) (emulator)
  macOS (desktop)             • macos         • darwin-arm64  • macOS 26.6.2 25G83
darwin-arm64

Checking for wireless devices...

No wireless devices were found.

Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to
diagnose potential issues. You may also try increasing the time to wait for
connected devices with the "--device-timeout" flag. Visit
https://flutter.dev/setup/ for troubleshooting tips.
```

Запуск демонстрационного приложения

Демонстрационное приложение hello_pam запущено на устройстве sdk gphone64 arm64 (emulator-5554), Android 15, API 35. При нажатии кнопки «+» значение счётчика увеличивается. Работа hot reload также проверена.

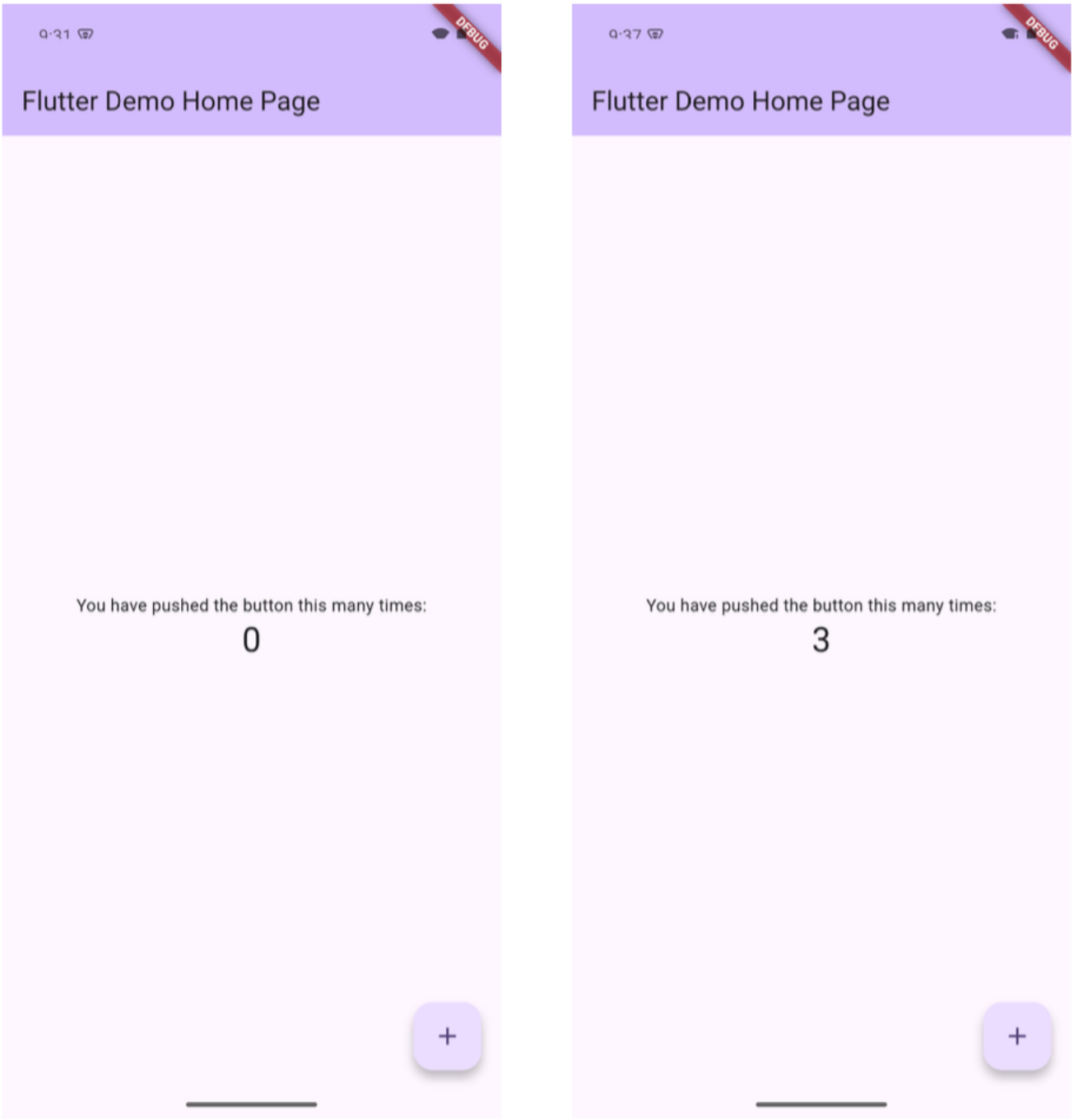


Рисунок 1 - работающий Android-эмулятор. Слева - начальное значение 0. Справа - значение 3 после трёх нажатий и hot reload.

Последовательность команд для создания и запуска

```
flutter create hello_pam
cd hello_pam
flutter run
```

Оценка состояния среды

Flutter, Android toolchain, подключённые устройства и сетевые ресурсы отмечены как работоспособные. flutter doctor сообщает о двух проблемных категориях: неполной установке Xcode с отсутствующим CocoaPods и отсутствии Chrome. Для выбранного Android-запуска эти компоненты не используются; iOS в задании является необязательной платформой. Общий вывод flutter doctor при этом не является полностью чистым.

Результат

Сборка, установка, запуск, обработка нажатий и сохранение состояния при hot reload подтверждены. Эти действия выполнены в demo hello_pam; FitTrack начнёт реализовываться на L2.

Репозиторий: https://github.com/yr3kdvdv/fittrack-pam

Часть B Упражнения на языке Dart

E1 Переменные и null safety

Функция parseOrDefault принимает строку или null и запасное целое значение. При null выполняется ранний возврат; иначе используется int.tryParse и оператор ?? для обработки неудачного преобразования. Оператор ! не используется.

Исходный код

```
// E1. Переменные и null safety.

int parseOrDefault(String? input, int fallback) {
  // int.tryParse принимает только String (не String?), поэтому null
  // отсекаем заранее. После этой проверки Dart сам «сужает» тип input
  // до String, и принудительное снятие nullability не нужно.
  if (input == null) return fallback;

  // tryParse не бросает исключение, а возвращает null для строк вроде
  // 'abc' или '7.5' - ?? подставляет fallback именно в этом случае.
  return int.tryParse(input) ?? fallback;
}

void main() {
  print("parseOrDefault('42', 0) → ${parseOrDefault('42', 0)}");
  print("parseOrDefault('abc', 0) → ${parseOrDefault('abc', 0)}");
  print("parseOrDefault(null, 7) → ${parseOrDefault(null, 7)}");
  print("parseOrDefault('7.5', -1) → ${parseOrDefault('7.5', -1)}");
}
```

Команда запуска

```
dart run dart_exercises/e1_null_safety.dart
```

Результат запуска

```
parseOrDefault('42', 0) → 42
parseOrDefault('abc', 0) → 0
parseOrDefault(null, 7) → 7
parseOrDefault('7.5', -1) → -1
```

E2 Коллекции и функциональные методы

Функция promotedAverage отбирает оценки не ниже 5 через where, сохраняет их в список, проверяет пустой результат и суммирует оценки через reduce. Деление суммы на количество даёт double. Явные циклы for и while не используются.

Исходный код

```
// E2. Коллекции и функциональные методы.

double promotedAverage(List<int> grades) {
  // where ленивый: без toList фильтр заново проходил бы по оценкам
  // при каждом обращении (isEmpty, reduce, length).
  final passed = grades.where((g) => g >= 5).toList();

  // reduce на пустом списке бросает StateError, поэтому пустой случай
  // обрабатываем до него - заодно это требование ТЗ «вернуть 0».
  if (passed.isEmpty) return 0.0;

  final sum = passed.reduce((a, b) => a + b);

  // Оператор / в Dart всегда возвращает double, даже для двух int,
  // поэтому явное приведение типа не нужно.
  return sum / passed.length;
}

void main() {
  print(
    'promotedAverage([9, 4, 7, 10, 3, 5]) → '
    '${promotedAverage([9, 4, 7, 10, 3, 5])}',
  );
  print('promotedAverage([10, 10, 9]) → ${promotedAverage([10, 10, 9])}');
  print('promotedAverage([3, 4, 2]) → ${promotedAverage([3, 4, 2])}');
}
```

Команда запуска

```
dart run dart_exercises/e2_collections.dart
```

Результат запуска

```
promotedAverage([9, 4, 7, 10, 3, 5]) → 7.75
promotedAverage([10, 10, 9]) → 9.666666666666666
promotedAverage([3, 4, 2]) → 0.0
```

E3 Именованные и необязательные параметры

Функция formatPrice принимает обязательную сумму amount и именованные параметры currency и decimals со значениями по умолчанию MDL и 2. Метод toStringAsFixed округляет число и формирует строку с нужным количеством десятичных знаков.

Исходный код

```
// E3. Функции с именованными и необязательными параметрами.

// Параметры в {} именованные: при вызове пишется имя (currency: 'EUR'),
// поэтому их можно передавать в любом порядке или не передавать вовсе -
// тогда берутся значения по умолчанию.
String formatPrice(double amount, {String currency = 'MDL', int decimals = 2}) {
  // toStringAsFixed округляет, а не отбрасывает знаки: 99.999 → "100.00".
  return '${amount.toStringAsFixed(decimals)} $currency';
}

void main() {
  print('formatPrice(12.5) → "${formatPrice(12.5)}"');
  print(
    "formatPrice(12.5, currency: 'MDL') → "
    '"${formatPrice(12.5, currency: 'MDL')}"',
  );
  print(
    "formatPrice(99.999, currency: 'EUR') → "
    '"${formatPrice(99.999, currency: 'EUR')}"',
  );
  print(
    "formatPrice(7.5, currency: 'USD', decimals: 1) → "
    '"${formatPrice(7.5, currency: 'USD', decimals: 1)}"',
  );
}
```

Команда запуска

```
dart run dart_exercises/e3_functions.dart
```

Результат запуска

```
formatPrice(12.5) → "12.50 MDL"
formatPrice(12.5, currency: 'MDL') → "12.50 MDL"
formatPrice(99.999, currency: 'EUR') → "100.00 EUR"
formatPrice(7.5, currency: 'USD', decimals: 1) → "7.5 USD"
```

E4 Класс Student

Класс Student хранит имя и группу в полях final. Именованный конструктор guest перенаправляет создание в основной конструктор. Равенство определяется по двум полям; hashCode вычисляется по этим же полям для корректной работы Set.

Исходный код

```
// E4. Классы.

class Student {
  final String name;
  final String group;

  Student(this.name, this.group);

  // Перенаправляем в основной конструктор, чтобы логика создания
  // была в одном месте, а guest() только задавал значения.
  Student.guest() : this('Guest', '-');

  @override
  String toString() => 'Student($name, $group)';

  // По умолчанию == сравнивает ссылки (один и тот же объект в памяти),
  // а нам нужно равенство по содержимому.
  @override
  bool operator ==(Object other) =>
      other is Student && other.name == name && other.group == group;

  // Set и Map сначала сравнивают hashCode, и только при совпадении
  // вызывают ==. Равные объекты обязаны иметь одинаковый hashCode,
  // поэтому считаем его из тех же полей, что участвуют в ==.
  @override
  int get hashCode => Object.hash(name, group);
}

void main() {
  print(Student('Ana Rusu', 'TI-231'));
  print(Student.guest());
  print(
    "Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231') → "
    '${Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231')}',
  );
  print(
    "{Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length → "
    '${{Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length}',
  );
}
```

Команда запуска

```
dart run dart_exercises/e4_classes.dart
```

Результат запуска

```
Student(Ana Rusu, TI-231)
Student(Guest, -)
Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231') → true
{Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length → 1
```

E5 Mixin Loggable

Mixin Loggable добавляет общий метод log классам CartService и AuthService через with. Метод выводит сообщение с фактическим типом объекта runtimeType. Оба сервиса демонстрируют логирование из своего метода.

Исходный код

```
// E5. Mixin.

// Mixin, а не базовый класс: у класса в Dart может быть только один
// родитель, а mixin-ов можно подключить сколько угодно. Так сервисы
// получают логирование, не тратя на него своё наследование.
mixin Loggable {
  // runtimeType - реальный класс объекта во время выполнения, поэтому
  // один и тот же код печатает CartService или AuthService.
  void log(String msg) => print('[$runtimeType] $msg');
}

class CartService with Loggable {
  void addItem(String item) => log('товар добавлен: $item');
}

class AuthService with Loggable {
  void login(String username) => log('пользователь вошёл: $username');
}

void main() {
  CartService().addItem('Paracetamol');
  AuthService().login('ana.rusu');
}
```

Команда запуска

```
dart run dart_exercises/e5_mixin.dart
```

Результат запуска

```
[CartService] товар добавлен: Paracetamol
[AuthService] пользователь вошёл: ana.rusu
```

E6 Асинхронное программирование

Функция fetchProfile возвращает Future<String> и имитирует загрузку задержкой в две секунды. await приостанавливает продолжение функции до завершения Future, не блокируя обработку других событий. В main профиль выводится перед сообщением «Готово.».

Исходный код

```
// E6. Асинхронное программирование.

Future<String> fetchProfile(String username) async {
  // Future.delayed не блокирует программу, а только планирует
  // продолжение через 2 секунды - так имитируется ответ сервера.
  await Future.delayed(const Duration(seconds: 2));
  return 'Профиль: $username, группа TI-231, решено упражнений: 42';
}

// main объявлен async, иначе внутри него нельзя использовать await.
Future<void> main() async {
  print('Загружается профиль пользователя ana.rusu...');

  // await приостанавливает main до готовности Future, поэтому
  // «Готово.» гарантированно печатается после профиля.
  final profile = await fetchProfile('ana.rusu');
  print(profile);

  print('Готово.');
}
```

Команда запуска

```
dart run dart_exercises/e6_async.dart
```

Результат запуска

Загружается профиль пользователя ana.rusu...
Профиль: ana.rusu, группа TI-231, решено упражнений: 42
Готово.
По зафиксированному построчному замеру первая строка появилась сразу, вторая и третья - через 2,01 с. Временные отметки не являются частью вывода самой программы.

Вывод по части B

Реализованы и проверены шесть упражнений: обработка nullable-значений, операции над коллекциями, именованные параметры, класс с равенством по значению, повторное использование поведения через mixin и асинхронное ожидание. Полученные результаты соответствуют примерам задания.

Приложение Спецификация проекта

Тема Т2 FitTrack Дневник тренировок

FitTrack - мобильное приложение для ведения личного дневника тренировок. Проект последовательно развивается в лабораторных работах L2–L6 и подключается к собственному backend.

Целевая аудитория и решаемая проблема

Приложение предназначено для студентов, которые регулярно тренируются и хотят отслеживать свою активность. Оно заменяет бумажные записи: хранит каталог упражнений, историю сессий, их длительность и заметки, позволяя видеть динамику занятий в телефоне.

Список экранов

1. Вход и регистрация - доступ к личному дневнику пользователя.
2. Каталог упражнений - поиск и фильтрация по группе мышц.
3. Карточка упражнения - описание, техника выполнения и изображение.
4. Дневник тренировок - список сессий в хронологическом порядке.
5. Форма новой сессии и редактирования - дата, упражнения, длительность и заметки.
6. Статистика прогресса - количество сессий по неделям и суммарная длительность.
7. Профиль пользователя - персональные данные учётной записи.

Сущности и основные поля

Поля совпадают с моделью данных темы. Типы предварительные, их сверю с backend на L5.

Exercise - упражнение из каталога.

Поле	Тип	Значение
id	String	идентификатор
name	String	название
muscleGroup	String	группа мышц, по ней работает фильтр
description	String	описание и техника
imageUrl	String	ссылка на изображение

WorkoutSession – одна тренировка в дневнике.

Поле	Тип	Значение
id	String	идентификатор
userId	String	владелец сессии
date	DateTime	дата тренировки
exerciseIds	List<String>	выполненные упражнения (ссылки на Exercise.id)
durationMin	int	длительность в минутах
notes	String	заметки

Почему выбрана эта тема

Выбрал FitTrack, потому что задача ведения тренировочного дневника имеет понятную практическую цель и позволяет последовательно освоить разработку мобильного приложения. Я сам спортсмен, поэтому для меня это приложение будет полезным не только как учебный проект. Мне важно видеть, сколько и как регулярно я тренируюсь, а дневник в телефоне удобнее записей, которые легко потерять. Делая FitTrack, я смогу сразу проверять его на своих тренировках и замечать, чего в нём не хватает.